

FlowGuard 5000

Pump Health Monitoring System

ENS 184 — Numerical Methods for Engineers | Mini-Project | Version 2

April 30, 2026

1. Project Overview

Your Mission

Welcome, engineers! Your mission is to design the **numerical engine** for a Predictive Maintenance (PdM) system that monitors a critical industrial pump — the **FlowGuard 5000**.

You will take raw, noisy sensor data and apply numerical methods to:

✓ **Calibrate** a pressure sensor (voltage → pressure mapping) using polynomial regression

✓ **Model** bearing vibration degradation over time using exponential curve fitting

✓ **Predict** when the bearing will fail, enabling proactive maintenance scheduling

✓ **Visualise** the pump’s health status in a professional dashboard

This is not a theoretical exercise. This is the exact workflow used by reliability engineers in aerospace, manufacturing, and energy sectors worldwide.

Step 1	Step 2	Step 3	Step 4
Load sensor CSV data (Student D)	Fit calibration polynomial: voltage → pressure (Student A)	Fit exponential degradation: hours → vibration (Student B)	Evaluate accuracy; predict failure hour; generate dashboard (Students C, D)

2. Team Formation & Roles

Each team of 4 students has specialised roles. **Assign roles during your first meeting and record them in the team contract.**

Student A — Linear Algebra Architect

Primary module: StudentA/interpolation.sci

Responsibility: Build the core polynomial regression engine. Your `poly_fit` and `eval_poly` functions form the foundation that calibrates the pressure sensor. You build the design matrix **Z** and apply the Normal Equations — the mathematics behind virtually every data-fitting tool in engineering software.

Deliverables:

- `poly_fit(x, y, k)` — least-squares polynomial fit
- `eval_poly(coeff, x_query)` — polynomial evaluation (Horner’s method)

Student B — Non-Linear Modelling Specialist

Primary module: `StudentB/differentiation.sci`

Responsibility: Implement log-linearisation to fit an exponential model to the vibration degradation data. Real bearing wear does not follow a straight line — it accelerates. Your model captures this behaviour and enables all failure predictions downstream.

Deliverables:

- `linearize_vib(hours, vibration)` — exponential fit by log-linearisation
- `predict_vibration(a, b, h_query)` — exponential prediction

Student C — Validation & Prognostics Analyst

Primary module: `StudentC/integration.sci`

Responsibility: A model is worthless without confidence metrics. You quantify how well Students A and B's models fit the data, and you implement the bisection root-finding algorithm that predicts the exact hour at which the pump bearing will fail.

Deliverables:

- `goodness_of_fit(y_actual, y_pred)` — RMSE and R^2 metrics
- `find_threshold_hour(hours, vib_fit, threshold)` — bisection failure prediction

Student D — Systems Integrator & Data Storyteller

Primary modules: `StudentD/load_data.sci`, `StudentD/health_dashboard.sci`, `StudentD/main.sce`

Responsibility: You are the glue that holds the project together. You load and validate the raw sensor data, compute the Health Index, generate the professional dashboard figure, and ensure that `main.sce` runs the full pipeline correctly end-to-end.

Deliverables:

- `load_data(fname)` — CSV I/O and data parsing
- `compute_health_index(vibration, threshold)` — HI normalisation
- `plot_report(...)` — two-panel health dashboard with zone lines
- `main.sce` — complete orchestration script

Important: Shared Responsibility

All members must contribute code. Roles define *primary* responsibility, not exclusive ownership. Every team member should understand every part of the codebase — you may be asked to explain any function during assessment.

3. Project Files & Folder Structure

All skeleton files are provided. Download them and organise as follows:

Required Folder Structure

```

Version2/
  pump_health_data.csv      <- sensor data (provided | do not modify)
  phm_project/
    main.sce                <- master script (Student D completes)
    data_loader/
      load_data.sci         <- Student D implements
    interpolation/
      interpolation.sci       <- Student A implements
    differentiation/
      differentiation.sci    <- Student B implements
    integration/
      integration.sci        <- Student C implements
    user_interface/
      health_dashboard.sci   <- Student D implements (D2 & D3)
    test_suites/
      run_all_tests.sci      <- run to check all modules
      test_studentA.sci
      test_studentB.sci
      test_studentC.sci
      test_studentD.sci
      test_integration.sci
  StudentA/
    interpolation.sci         <- YOUR implementation file (Student A)
  StudentB/
    differentiation.sci      <- YOUR implementation file (Student B)
  StudentC/
    integration.sci          <- YOUR implementation file (Student C)
  StudentD/
    load_data.sci            <- YOUR implementation file (Student D)
    health_dashboard.sci     <- YOUR implementation file (Student D)
    main.sce                 <- YOUR completed script (Student D)

```

Important: All `.sci` files in `phm_project/` are *skeleton code*. They contain function signatures and `// TODO` comments but will not produce correct results until you implement the logic. Your implementation goes into your individual `StudentX/` file.

4. Two-Week Timeline

Days 1–3: Team Setup & API Agreement

Deliverable: Signed team contract uploaded to Google Classroom.

- Read the Project Manual (PDF) and Backgrounder Notes (PDF).
- Form your team of 4 and assign the roles above.
- Complete `team_contract_template.md`: agree on the exact function signatures, what each function will print, and how you will test integration.
- All members sign off on the contract.
- Student D: run `exec('main.sce', -1)` and confirm the skeleton loads without errors (it will show “TODO” placeholder messages).

Days 4–7: Individual Implementation

Deliverable: Each member uploads a screenshot showing their function passing at least one self-designed test.

- Students A, B, C: implement your two functions in your `StudentX/` skeleton file.
- Test with simple known data before testing with the real CSV:
 - **Student A:** `poly_fit([1;2;3],[2;4;6],1)` should give `[0; 2]` (line $y = 2x$).
 - **Student B:** `linearize_vib([0;1;2],[1;e;e^2])` should give $a \approx 1$, $b \approx 1$.
 - **Student C:** `goodness_of_fit([1;2;3],[1;2;3])` should give $\text{RMSE} = 0$, $R^2 = 1$.
 - **Student D:** `load_data(fname)` should return 4 vectors each of length 120.
- Run the individual test suite: `exec('test_suites/test_studentX.sci', -1)`.

Days 8–9: Integration & Team Debugging

Deliverable: Working `main.sce` that runs end-to-end without errors.

- Student D copies the completed functions into `phm_project/` and runs `exec('main.sce', -1)`.
- Run the full test suite: `exec('test_suites/run_all_tests.sci', -1)`. All tests must report PASS.
- Whole team: collaborative debugging session. Log any errors in `progress_tracker.md`.

Days 10–12: Analysis & Report Writing

Deliverable: Draft report shared with the team for peer review.

- Generate the final health dashboard PNG.
- Each member writes their assigned report section (see Section 5).
- Discuss as a team: Do the model results make physical sense? Are the R^2 values acceptable? Is the predicted failure hour reasonable?

Days 13–14: Final Review & Submission

Deliverable: Single ZIP submitted to Google Classroom.

- Compile the final PDF report.
- Complete `submission_checklist.md`.
- Package into `Team[X]_FlowGuard5000.zip`.
- Submit before the deadline.

5. Submission Requirements

Submit a single ZIP file named `Team[X]_FlowGuard5000.zip` (replace `[X]` with your team number).

The ZIP must contain:

File	Notes
StudentA/interpolation.sci	Fully implemented
StudentB/differentiation.sci	Fully implemented
StudentC/integration.sci	Fully implemented
StudentD/load_data.sci	Fully implemented
StudentD/health_dashboard.sci	Fully implemented
StudentD/main.sce	Complete pipeline
pump_health_data.csv	Unmodified original
pump_health_report_v2.png	Generated by your code
Final_Report.pdf	See report outline below
submission_checklist.md	All boxes ticked
team_contract_template.md	Signed, from Day 1–3

Report outline (each member writes their section):

1. **Introduction** — problem statement and team roles
2. **Student A** — polynomial calibration: method, coefficients, calibration curve plot, R^2
3. **Student B** — exponential degradation model: method, a and b values, fitted curve plot
4. **Student C** — model validation: RMSE and R^2 for both models, failure hour prediction, bisection convergence
5. **Student D** — health dashboard: HI interpretation, dashboard figure, maintenance recommendation
6. **Conclusion** — key findings and engineering recommendation for the FlowGuard 5000

6. Success Criteria (Grading)

Grading Breakdown

Category	Weight	What We Are Looking For
Functionality	40%	main.sce runs without errors end-to-end. Correct calibration coefficients, exponential model parameters, and failure hour prediction. PNG output generated.
Algorithmic Correctness	25%	Design matrix Z built correctly. Use the backslash operator (e.g. A \ B), not inv() . Log-linearisation math correct. Bisection converges to the correct bracket.
Code Quality	15%	Modular, well-commented code. Input validation where appropriate. Meaningful variable names. No hard-coded magic numbers.
Report & Analysis	20%	Clear dashboard with proper labels and legends. Discussion of model quality (R^2 , RMSE). Correct interpretation of the Health Index. Engineering recommendation supported by data.

7. Rules & Academic Integrity

Rules — Read Carefully

- **No built-in curve fitting functions.** You may **not** use `polyfit()`, `reglin()`, `lsq()`, or any similar black-box fitting function. The point is to build the numerical engine from scratch.
- **Use the backslash operator, not `inv()`.** Solve $Ax = b$ with `A \ b`. Using `inv(A) * b` is numerically inferior and **will result in a deduction**.
- **Allowed Scilab built-ins:** the backslash operator (`\`), `log`, `exp`, `sqrt`, `sum`, `mean`, `length`, `zeros`, `ones`, `linspace`, all plotting functions, `read_csv`, `evstr`, `isfile`, `mopen/mclose`.
- **All members must contribute meaningfully.** Peer evaluations will be conducted. Division of labour is part of the learning objective.
- **Cite all external references** in your report (textbooks, lecture notes, online resources). AI-generated code submitted without understanding or citation is a violation of academic integrity policy.

8. Frequently Asked Questions

FAQ

Q: Can I use Python or MATLAB instead of Scilab?

A: No. The project is designed for Scilab. Your code will be tested in Scilab. All functions must be in `.sci/.sce` files.

Q: What does “log-linearisation” mean — do I need calculus?

A: No calculus needed. It means taking `log()` of both sides of the exponential model equation to convert it into a form that can be fitted with the same Normal Equations as the polynomial. The Backgrounder Notes walk through this step-by-step.

Q: My main.sce shows an error — what do I do?

A: Read the error message carefully — it tells you the file name and line number. Check that (a) the data file path is correct, (b) your functions return the expected output sizes, and (c) you are not passing row vectors where column vectors are expected (`x = x(:)` forces a column).

Q: What if I don't finish my module on time?

A: Communicate with your team immediately. Integration (Days 8–9) depends on all modules being ready. A non-functional placeholder (returning zeros) can unblock integration while you finish the real implementation.

Q: Can one person write all the code?

A: No. Peer evaluations and individual code reviews will be conducted. Every member must be able to explain their own module in detail.

Q: How do I test my function before Student D integrates everything?

A: Run your individual test suite directly:

```
exec('test_suites/test_studentA.sci', -1);    // for Student A
exec('test_suites/test_studentB.sci', -1);    // for Student B
exec('test_suites/test_studentC.sci', -1);    // for Student C
exec('test_suites/test_studentD.sci', -1);    // for Student D
```

Also design your own tests with simple hand-checkable data before testing with the real 120-point CSV.

Q: What is the `h_star = %inf` message?

A: It means the bisection found no sign change — the fitted vibration curve does not cross the threshold within the available data range. This is a code bug if the data clearly shows the threshold being crossed; check `max(vib_exp)` vs. `THRESHOLD` to diagnose.

Q: What if `main.sce` doesn't run on the first integration attempt?

A: This is expected! Integration is where real engineering happens. Debug collaboratively, use `disp()` statements to trace variable values, and log issues in `progress_tracker.md`.

9. Getting Started Checklist

Before You Write a Single Line of Code

- ☐ Read the Project Manual PDF (theory, task descriptions, expected outcomes).
- ☐ Read the Backgrounder Notes PDF (intuitive explanations, worked examples).
- ☐ Form your team of 4 and assign the four roles.
- ☐ Complete and sign the Team Contract (Days 1–3 deliverable).
- ☐ Set up the project folder structure as shown in Section 3.
- ☐ Verify Scilab opens without errors on your machine.
- ☐ Run: `exec('phm_project/main.sce', -1)` — the skeleton should load and print “TODO” messages without crashing.
- ☐ Confirm `pump_health_data.csv` is in the `Version2/` folder (run the instructor data generator if not).

Implementation Checklist (per student)

- ☐ Understand the mathematical method (read Backgrounder Notes Section for your role).
- ☐ Implement functions in your `StudentX/` file.
- ☐ Test with simple hand-checkable data first.
- ☐ Run your individual test suite and achieve all PASS.
- ☐ Copy your completed file into `phm_project/` for integration.
- ☐ Run `exec('test_suites/run_all_tests.sci', -1)` — all PASS.
- ☐ Run `exec('phm_project/main.sce', -1)` — no errors, PNG generated.

“The goal is not just to get the answer — it is to understand why the method works and when to apply it.”

Good luck, engineers!

Questions? Post them in the Google Classroom comments.